

ESERCIZI SOLO SULLA SECONDA PARTE DEL CORSO:

- COMPLESSITA'
- STRUTTURE DATI

1

Definire un algoritmo MAXPITAGORA che prende in input un array A di elementi interi positivi (strettamente maggiori di 0) e, se esiste in A almeno un valore tale che il suo quadrato è la somma dei quadrati di altri 2 valori anch'essi presenti in A , restituisce il massimo di tali valori; altrimenti (se nessun elemento di A è tale che il suo quadrato è la somma dei quadrati di altri 2 elementi di A) restituisce 0. Si valuti la complessità dell'algoritmo ideato.

SOLUZIONE:

In altre parole, voglio cercare tutte le terme a, b, c tali che $c^2 = a^2 + b^2$, e voglio restituire il massimo c .

SOLUZIONE ESAUSTIVA:

Enumero tutte le terme e scelgo il c massimo.

$\max \leftarrow 0$

for $i = 1$ to n

for $j = i+1$ to n

for $k = j+1$ to n

if $c^2 = a^2 + b^2$

if $c > \max$

$\max \leftarrow c$

return $\max$

Devo fare tre "or",
in cui cambio
la c ogni volta

$$\text{COMPLESSITA'} = \sum_{i=1}^n \sum_{j=i+1}^n \sum_{k=j+1}^n 1 = \Theta(n^3),$$

dove n è la dimensione di A

Sicuramente si può fare di meglio ...

SOLUZIONE MIGLIORE:

Usiamo un algoritmo simile a quello per trovare in un array due valori la cui somma è k :

- Ordino l'array e elevo tutti gli elementi al quadrato
- Per ogni coppia di elementi (x, y) , faccio una ricerca binaria di un z tale che $z = x + y$

→ La complessità dell'algoritmo è $O(n^2 \log(n))$, perché è dominata dai due cicli per elencare le coppie (x, y) (n^2) e la complessità dell'algoritmo di ricerca binaria ($\log(n)$)

Ma si può fare ancora meglio?

SOLUZIONE ANCORA MIGLIORE:

Usiamo una struttura dati comoda, per esempio, usiamo una tabella hash, che è comoda per gli inserimenti, le ricerche e le rimozioni

Infatti, se la HT è fatta "bene", la ricerca nelle tabelle hash è un $O(1)$, mentre gli inserimenti e le rimozioni sono $O(n)$ → Complessità = $O(n^2)$

SOLUZIONE UGUALE MA NON RICHIESTA ALL'ESAME:

- Ordino l'array
- Prendo un solo elemento dell'array $A[i]$.

Se esiste una terma pitagorica con $c = A[i]$, allora esistono j e k t.c. $A[j]^2 + A[k]^2 = A[i]^2$, e questi j e k saranno indici compresi tra 1 e $i-1$.

IDEA: prendo $j=1$ e $k=i-1$ e calcolo $A[j]^2 + A[k]^2$;

- Se il risultato è minore di $A[i]^2$, allora incremento j
- Se il risultato è maggiore di $A[i]^2$, allora decremento k
- Se il risultato è uguale a $A[i]^2$, ho trovato la terma con c massimo $\Rightarrow$ FINE
- Se ho $j = k$, allora non ho trovato una terma $\Rightarrow$ decremento i
- Se ho $i < 3$, allora non esiste alcuna terma $\Rightarrow$ STAMPO ZERO

Anche qui la complessità è $O(n^2)$, perché per ogni i , cercare se esiste una terma costa $O(n)$, e questa cosa la ripeto per ogni elemento dell'array.

2

Si realizzi un algoritmo che, dati n array ordinati di lunghezza m , realizzi un unico array ordinato contenente tutti gli elementi. Si calcoli la complessità temporale dell'algoritmo in funzione di n e m .

1^a IDEA:

Concateniamo tutti gli array e facciamo il MERGESORT
COMPLESSITA' = $O(n \cdot m \cdot \log(n \cdot m))$

2^a IDEA:

Ci è detto che gli array sono ordinati. Quindi possiamo procedere così:

Guardo il primo di ogni array e metto il più piccolo nell'array finale. Poi incremento l'indice su quell'array e ripeto fino a quando tutti gli indici sono arrivati alla fine dei loro rispettivi array

COSTO TOTALE = $\Theta(n^2 \cdot m)$ ($n \cdot m$ perché visito tutti gli elementi, n per la ricerca del minimo)

Questa idea può funzionare se m è molto grande e n è molto piccolo. In caso contrario, conviene usare la prima strategia

DOMANDA: È possibile trovare qualcosa che è il meglio dei due mondi?

3^a IDEA:

Ricordiamo che la ricerca del minimo ha costo ridotto in un MINHEAP ($\Theta(1)$).

Quindi prendo il primo elemento di ogni array e lo inserisco nel MINHEAP (quindi costruisco

un array lungo n ($O(n)$) in cui faccio MINHEAPIFY dei suoi n elementi. Il minimo è il primo elemento dell'array). Ogni volta, rimuovo il minimo ($O(\log(n))$), e inserisco il successivo di quell'elemento nell'array originale. Ripeto fino a quando non ho inserito e rimosso tutti gli elementi dagli array originali. La complessità finale è $O(n \cdot m \cdot \log(n))$

visito tutti gli elementi degli array

NOTA:

Si poteva usare anche un RBT: la ricerca del minimo costa, la rimozione di un nodo e l'inserimento di un nodo costano tutti $O(\log(n))$. Quindi, la complessità asintotica rimane la stessa, ma la ricerca del minimo è più veloce in un MINHEAP. Dunque, l'algoritmo sarebbe più efficiente usando un MINHEAP.

3

Un "multi-insieme di interi" è una collezione che si comporta come un insieme di interi, ma ammette duplicati e tiene traccia del numero di occorrenze di ciascun intero. Si richiede di descrivere in pseudocodice le seguenti procedure, dove M è un multi-insieme e n è un intero, fornendone anche le complessità asintotiche nel caso medio:

- 1 INSERT(M, n), che inserisce n in M e restituisce il numero di occorrenze di n in M dopo l'inserimento;
- 2 DELETE(M, n), che cancella n da M e restituisce il numero di occorrenze di n in M dopo la cancellazione (restituisce -1 se n è assente);
- 3 COUNT(M, n), che restituisce il numero di occorrenze di n in M .

SOLUZIONE:

Per prima cosa, utilizziamo una struttura dati "comoda" per implementare queste procedure. Per esempio, si potrebbe usare una HASH TABLE a indirizzamento aperto: la chiave per calcolare l'hash è n , ma all'interno del "bucket" c'è anche un altro valore: il numero di occorrenze di n .

Quindi...

- ① Se voglio inserire n , calcolo la sua hash e cerco il "bucket". All'interno del bucket, incremento il contatore e lo ritorno.
- ② In modo analogo faccio la rimozione. Però, se trovo il contatore uguale a zero, significa che non ho n da rimuovere, quindi ritorno -1 .
- ③ In questo caso semplicemente restituisco il valore del contatore.

Se la HT è fatta "bene", le operazioni di inserimento, ricerca e rimozione costeranno $\Theta(1)$, che sarà la complessità delle tre procedure.

4

Risolvere le seguenti ricorrenze:

$$① \quad T(n) = 3T\left(\frac{n}{2}\right) + n$$

$$② \quad T(n) = 16T\left(\frac{n}{4}\right) + n!$$

$$③ \quad T(n) = \frac{1}{2}T\left(\frac{n}{2}\right) + n^{-1}$$

$$④ \quad T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

SOLUZIONE:

In tutti e quattro i casi ci è suggerito provare a applicare il Master Theorem

$$① \quad a = 3; \quad b = 2; \quad f(n) = n = O(n^{\log_b(a)})$$

$$\Rightarrow \text{Stiamo nel caso } ① \Rightarrow T(n) = \Theta(n^{\log_2(3)})$$

$$② \quad a = 16; \quad b = 4; \quad f(n) = n! = \Omega(\underbrace{n^{\log_b(a)}}_{n^2})$$

$\Rightarrow$ Stiamo nel caso ③ $\Rightarrow$ Si deve verificare la condizione di regolarità, cioè $\exists c$ t.c.

$$af\left(\frac{n}{b}\right) \leq cf(n) \Rightarrow 16 \cdot \left(\frac{n}{4}\right)! \leq cn!$$

Se per esempio prendo $c = 16$, ottengo $n! \geq \left(\frac{n}{4}\right)!$, che è certamente vero $\Rightarrow$ la condizione di regolarità è rispettata $\Rightarrow$ posso applicare il Master Theorem nel caso ③ $\Rightarrow T(n) = \Theta(n!)$

NOTA: La condizione di regolarità tipicamente non è rispettata quando in $f(n)$ c'è un seno o coseno,

i quali "oscillano" tra due valori.

In questi casi conviene dire che $\cos(n) \leq 1$ e trovare un limite asintotico superiore senza il coseno

③ Nelle ipotesi del Master Theorem, è necessario che $a \geq 1 \rightarrow$ Non posso applicare il Master Theorem

Non si può nemmeno applicare l'albero di ricorsione, perché a dovrebbe essere un intero (come fai a dire come si dividono i rami?)

Ci resta solo il metodo di sostituzione. Ma per applicarlo ci serve una ipotesi. Proviamola facendo l'UNFOLDING...

$$\begin{aligned} T(n) &= \frac{1}{2} T\left(\frac{n}{2}\right) + \frac{1}{n} = \frac{1}{2} \left[\frac{1}{2} T\left(\frac{n}{4}\right) + \frac{2}{n} \right] + \frac{1}{n} = \\ &= \frac{1}{4} T\left(\frac{n}{4}\right) + \frac{2}{n} = \frac{1}{4} \left[\frac{1}{2} T\left(\frac{n}{8}\right) + \frac{4}{n} \right] + \frac{2}{n} = \\ &= \frac{1}{8} T\left(\frac{n}{8}\right) + \frac{3}{n} = \dots \end{aligned}$$

$\Rightarrow$ Ogni volta che calcolo $T(\dots)$, si aggiunge un termine $\frac{1}{n}$ e l'input viene diviso per 2 $\Rightarrow$ Al massimo, dividerò l'input $\log_2(n)$ volte, e quindi avremo $\log_2(n)$ volte il termine $\frac{1}{n}$

$\Rightarrow$ Ci viene da dire che $T(n) = \Theta\left(\frac{1}{n} \log(n)\right)$

Verifichiamo questa cosa per sostituzione...

• Verifico che $T(n) = O\left(\frac{1}{n} \log(n)\right)$, cioè che
 $\exists c$ t.c. $T(n) \leq c \cdot \frac{1}{n} \log(n)$

$$\text{IPOTESI: } T\left(\frac{n}{2}\right) \leq c \cdot \frac{1}{\frac{n}{2}} \log\left(\frac{n}{2}\right) = \frac{2c}{n} (\log n - \log 2)$$

$$\begin{aligned} T(n) &= \frac{1}{2} T\left(\frac{n}{2}\right) + \frac{1}{n} \leq \frac{1}{2} \cdot \frac{2c}{n} (\log n - \log 2) + \frac{1}{n} = \\ &= c \cdot \frac{\log n}{n} - \frac{1}{n} (1 - c \log 2) \leq c \cdot \frac{\log n}{n} \Rightarrow \end{aligned}$$

$$\rightarrow 1 - c \log 2 \leq 0 \quad \rightarrow c \geq \frac{1}{\log 2}$$

$\uparrow$
 $n > 0$

Quindi c esiste e dunque $T(n) = O\left(\frac{1}{n} \log(n)\right)$

• Verifico che $T(n) = \Omega\left(\frac{1}{n} \log(n)\right)$

I passaggi sono uguali a quelli di prima,
ma con il segno della disuguaglianza invertito

$$\text{Quindi abbiamo che } c \leq \frac{1}{\log 2}$$

$$\text{Quindi } c \text{ esiste } \rightarrow T(n) = \Omega\left(\frac{1}{n} \log(n)\right)$$

$$\text{Dunque } T(n) = \Theta\left(\frac{1}{n} \log(n)\right)$$

④ $a = 4$; $b = 2$; $f(n) = n^2 = \Theta(n^{\log_b(a)})$

→ CASO ② del Master Theorem

→ $T(n) = \Theta(n^2 \log(n))$

⑤ Si consideri il linguaggio $L = \{a^n(bc)^n \mid n \geq 2\} \cup \{b^n(ca)^n \mid n \geq 1\} \cup (abc)^+$. Si descrivano con sufficiente precisione una macchina di Turing a nastro singolo, una MT ad un nastro e una RAM che accettino L e se ne valutino le complessità spaziali e temporali, utilizzando il criterio di costo più opportuno.

SOLUZIONE:

E' facile dedurre che stiamo nel secondo insieme se il primo carattere che leggo e' una 'b'.

Invece, se leggo due 'a' consecutive allora sto nel primo insieme. Infine, se leggo 'a' e poi 'b', sto nel terzo insieme. A questo punto, basta verificare che valgono le proprietà di quell'insieme

① MT NASTRO SINGOLO

• Primo insieme

Per verificare che il numero di lettere sia uguale in una MT a nastro singolo bisogna cancellare una lettera a ogni passaggio e verificare che alla fine me ho zero

ESEMPIO : $aaabcbcbcb \longrightarrow \cancel{a}a\cancel{a}b\cancel{c}bcbcb \longrightarrow$
 $\longrightarrow \cancel{a}\cancel{a}b\cancel{c}\cancel{b}bcbcb \longrightarrow \cancel{a}\cancel{a}\cancel{b}\cancel{c}\cancel{b}\cancel{c}bcbcb \longrightarrow$
 $\longrightarrow$ Analizzo un'ultima volta la stringa per verificare che non ci sia nessun carattere

COMPLESSITA' :

- Temporale : In una stringa accettabile, il # di 'a' e' uguale a $\frac{1}{3}$ della stringa $\rightarrow \frac{n}{3}$
per ogni 'a' che trovo, devo ripercorrere tutta la stringa $\rightarrow n \cdot \frac{n}{3} = O(n^2)$
- Spaziale : oltre che all'input, non si usa spazio ausiliario $\rightarrow O(n)$
- Secondo insieme
Analogo al primo $\rightarrow$ complessita' invariate
- Terzo insieme
E' un linguaggio regolare $\rightarrow$ basta una passata della stringa $\rightarrow$ la complessita' temporale diventa $O(n)$

② MT A UN NASTRO

In questo caso, si puo' usare il nastro di memoria come una pila dove teniamo conto del numero di occorrenze delle lettere

$\rightarrow$ COMPLESSITA' :

- Temporale = $O(n)$: faccio un solo passaggio dell'input
- Spaziale = $O(n)$, perche' devo impilare tutte le 'a' (o le 'b'), che abbiamo detto essere $\frac{n}{3}$ della stringa. Se contassimo in binario, la complessita' spaziale puo' scendere a $O(\log(n))$, mentre quella temporale rimane invariata

③ MACCHINA RAM

il funzionamento è analogo a prima, ma in questo caso il contatore è un intero contenuto in una cella di memoria

COMPLESSITA' (CON CRITERIO DI COSTO COSTANTE):

- $T(n) = O(n)$
- $S(n) = O(1)$

COMPLESSITA' (CON CRITERIO DI COSTO LOGARITMICO):

- $T(n) = O(n \log(n))$, perché si aggiunge il costo delle addizioni sul contatore
- $S(n) = O(\log(n))$, cioè la dimensione in bit del contatore

DOMANDA: Qual è il criterio più opportuno?
Poiché il contatore non aggiunge troppo costo al costo logaritmico, va benissimo usare quello di costo costante

Il criterio di costo logaritmico va usato quando il costo aumenta tanto rispetto a quello del criterio di costo costante!
